Traffic Forwarding and Redirection in Mininet SDN Topology With Ryu Controller Framework

Jinlong.Huang
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Jinlong.Huang23@student.xjtlu.edu.cn
2364343

Leyi.Li
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Leyi.Li23@student.xjtlu.edu.cn
2362141

Nuoqian.Xu
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Nuoqian.Xu23@student.xjtlu.edu.cn
2363177

Shuheng.Hu
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Shuheng.Hu23@student.xjtlu.edu.cn
2362132

Zeyu.Yin
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Zeyu.Yin23@student.xjtlu.edu.cn
2360955

***Abstract*—This project uses the Mininet network emulator and the Ryu SDN controller to design and evaluate a basic SDN architecture that overcomes limitations of traditional networks. It builds a custom SDN topology, simulates traffic control, and develops a controller application for selective traffic forwarding and service redirection.**

Key tasks include installing and managing flow entries, analyzing traffic, and measuring latency to ensure efficient and transparent client–server redirection. The design is documented with diagrams, pseudo-code, and flowcharts. Experiments are conducted to validate performance, and possible future improvements and extensions are discussed.

***Index Terms*—SDN, Ryu controller, Mininet, Forwarding, Redirecting, TCP/IP, Load balancing**

I. Introduction

A. Background

Software-Defined Networking (SDN) is a paradigm shift in network architecture that decouples the control plane from the data planes. In conventional IP networks, forwarding functions and control logic are integrated within individual network devices, constraining flexibility and programmability. By contrast, SDN introduces a logically centralized controller that exposes programmatic interfaces, enabling operators to configure, monitor, and optimize network behavior via software and to adapt forwarding policies dynamically in response to changing conditions.

B. Task Specification

The project can be divided into three parts:

1) Utilize the Mininet tool to construct the network topology, set up the accurate IP and MAC addresses for all network nodes, and verify that the basic network connectivity (e.g., node-to-node communication) is fully functional.

2) Develop an SDN controller application with the Ryu framework to implement traffic forwarding and SDN flow table management functionalities (e.g., creating, installing, and configuring flow entries).

3) Implement key functions such as traffic redirection (from the target server to another server) and network latency measurement to analyze and compare test results.

C. Project Challenges

While achieving project objectives, there also exists some challenges remain to be solved:

1) In order to ensure optimal functionality and prevent performance degradation caused by frequent updates, it is necessary to generate, update and delete flow table entries dynamically, based on real-time network traffic.

2) The controller must adjust packet headers (dest IP/MAC) when redirecting to Server2, ensuring uncorrupted TCP sequence numbers for intact handshakes.

3) Mininet virtual link packet loss may mimic controller bugs. The challenge is using switch port stats to distinguish Mininet limitations from flow entry issues.

D. Contributions

In order to complete the project successfully, it is essential that all group members understand and master the core concepts of SDN. In addition, we will learn about the Ryu controller programming framework and master the implementation of event handling and dynamic deployment of flow table entries. This project integrates SDN theory, network simulation, controller programming, algorithm design and network performance analysis to comprehensively enhance understanding of and practical skills in SDN technologies.

II. Related Work

Software Defined Networking (SDN) is a network architecture that simplifies management by separating the control

plane from the data plane. This separation enables dynamic resource management, enhances flexibility and programmability, and supports advanced traffic engineering [1].

Nginx [2] is an open-source web server and reverse proxy. It also redirects network traffic by distributing and balancing client requests across multiple servers in a cluster, providing a practical example of traffic redirection.

With the rapid growth of streaming multimedia services, traditional Content Delivery Networks (CDNs) face challenges such as difficulty in guaranteeing high Quality of Experience (QoE) and increasing infrastructure costs. SDN offers new strategies to address these issues, for example by locating the content server closest to a user and redirecting that user's requests to it. This improves both energy efficiency and user experience, as reported by Yang et al. [3].

This redirection is transparent to the user, who does not know which physical server is actually processing the request. From the user's perspective, they simply receive a normal response—for instance, the client appears to be communicating with server1, while the communication is in fact handled by server2.

III. Design

A. *Design diagram*

Fig.1 shows the SDN network topology constructed by this project.

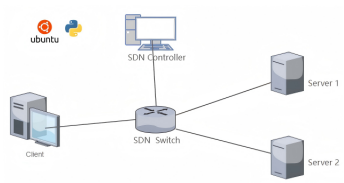

Fig. 1. Client-Server Application

As shown in the figure, two servers and a client are connected to the switch, whose function is to forward the packets sent by the host to the specified address. Before forwarding the packet, the switch compares the flow table and records the event to the SDN controller, which then allows the server to tell the switch the destination address of the packet based on protocols such as OpenFlow.

B. *Workflow*

Fig.2 and Fig.3 show sequential diagrams of traffic forwarding and redirection respectively, to help better illustrate the workflow.

In this forwarding scenario, the process begins when the client transmits a TCP SYN packet to Server 1. The switch detects this packet, triggers a packet-in event, and redirects it to the SDN controller. After examining the packet's header details, the controller determines that the traffic should be routed straight to Server 1. It then installs the necessary flow table entries on the switch by issuing flow control messages. The idle timeout is set to be 5 seconds, enabling

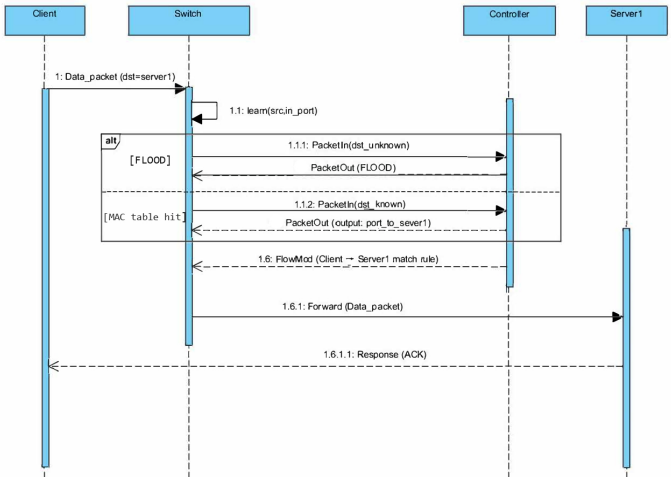

Fig. 2. Sequence diagram of forwarding

automatic removal when the connection remains inactive for that duration.

Once the rules are in place, the switch uses the updated flow table to forward any following TCP SYN packets directly to Server 1. In response, Server 1 sends back a SYN-ACK packet, which the switch passes on to the client. The client completes the three-way handshake by returning an ACK packet. From this point forward, all further communication between the client and Server 1 don't require additional controller intervention.

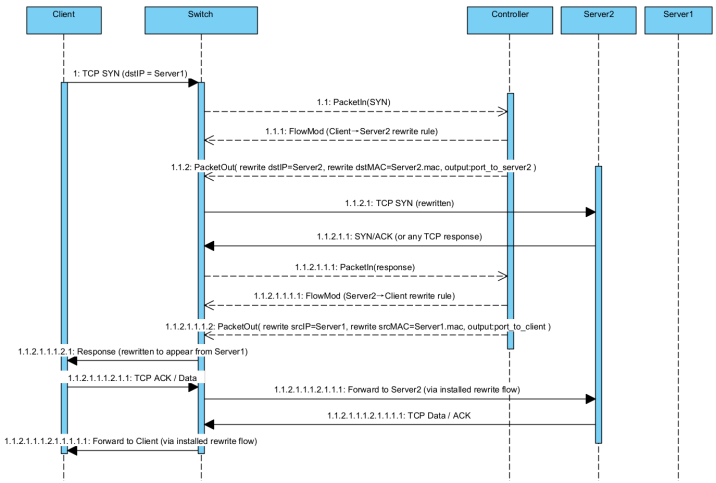

Fig. 3. Sequence diagram of redirection

In a redirection scenario, the Client first sends an initial TCP SYN packet to Server1, and the switch reports this packet to the controller as a `Packet-In`. After the controller verifies that the packet is an IPv4/TCP SYN from the preconfigured Client to Server1, it determines that the connection needs to be redirected. It installs a forward flow entry on the switch through `Flow-Mod`, using the TCP five-tuple (source/destination IP, source/destination port, and protocol number) as the match condition. The action rewrites the packet's destination

IP and MAC from Server1 to Server2 and forwards it to the port leading to Server2. This flow entry has a 5-second `idle_timeout` for automatic deletion when the connection is idle.

Subsequently, when the switch reports the first response packet returned from Server2 to the Client to the controller, the controller installs a corresponding reverse flow entry for the same TCP five-tuple. This entry rewrites the source IP and MAC from Server2 to Server1 and outputs it to the port where the Client is located, also with a 5-second `idle_timeout`. The first SYN and the first response packet are rewritten and forwarded through `Packet-Out` according to the above rules, completing the three-way handshake between the Client and Server2.

C. *Algorithm*

The algorithms for traffic forwarding and traffic redirection are as follows:

Algorithm 1: Ryu Forwarding Algorithm

```
Input: packet-in event
src ← packet.src;
dst ← packet.dst;
in_port ← packet.in_port;
mac_table[src] ← in_port
{self-learn:} mac_to_port[dpid][src] ← in_port;
if dst exists in mac_to_port[dpid] then
  | out_port ← mac_table[dst];
end
else
  | out_port ← FLOOD;
end
if out_port ≠ FLOOD then
  | add_flow(match(src, dst, in_port), out_port);
end
send_packet(out_port);
```

IV. Implementation

A. *Development Environment*

During the project, the development was primarily carried out on the computer listed in Table I.

TABLE I
Development Environment

	Specification
CPU	13th Gen Intel(R) Core(TM) i9-13900HX
Memory	16GB
Operating System	Ubuntu 22.04.5 LTS
Python IDE	PyCharm 2024.3
Python version	Python 3.10.12
SDN controller	Ryu 4.34

B. *Programming Skills*

- **Ryu/OpenFlow coding skill**:

Algorithm 2: Ryu Redirect Switch Logic

```
Input: PacketIn message msg
Output: Forward, flood or redirect action
pkt ← parse(msg.data)
eth ← pkt.eth
in_port ← msg.in_port
if eth.type = LLDP then
  | return
end
Learn(src_mac → in_port)
if eth.type = ARP then
  | Flood(pkt)
  | return
end
if dst_mac known then
  | out_port ← learned port
else
  | out_port ← FLOOD
end
if out_port = FLOOD then
  | Flood(pkt)
  | return
end
ip ← pkt.IPv4
tcp ← pkt.TCP
if ip and tcp exist then
  | ← tcp.flags
  | Client → Server1 (TCP SYN) if ip.src =
  | CLIENT.ip and ip.dst = SERVER1.ip and SYN flag
  | set then
  |   | port ← port of SERVER2 (or FLOOD)
  |   | InstallFlow( match = 5-tuple(Client→S1),
  |   | actions = rewrite(dstIP=SERVER2.ip,
  |   |   dstMAC=SERVER2.mac), output(port),
  |   | priority=10, idle_timeout=5 )
  |   | PacketOut(actions)
  |   | return
  | end
  | Server2 → Client (return) if ip.src = SERVER2.ip
  | and ip.dst = CLIENT.ip then
  |   | port ← learned port of CLIENT
  |   | InstallFlow( match = 5-tuple(S2→Client),
  |   | actions = rewrite(srcIP=SERVER1.ip,
  |   |   srcMAC=SERVER1.mac), output(port),
  |   | priority=10, idle_timeout=5 )
  |   | PacketOut(actions)
  |   | return
  | end
end
Install normal learning-switch flow
Forward(pkt, out_port)
```

- Use `@set_ev_cls` to handle OFPSwitchFeatures and OFPPacketIn events;
- Be able to initially understand and use the rules such as flow table, priority, match fields, actions, buffer_id, idle_timeout in OpenFlow

- **Headline parsing and flow table construction**:
  - Use the packet module to parse the Ethernet, IPv4 and TCP headers
  - Based on the header fields, construct the matching logic to achieve TCP-based traffic redirection, and realize the transparent effect during the traffic redirection process
- **OOP (Object-Oriented Programming)**:
  - The controller logic is organized using an object-oriented approach. By inheriting from `RyuApp` and encapsulating states and behaviors (such as flow installation and packet-in handling) within the controller class, the modularity, readability, and reusability of the code are improved.

C. Steps of Implementation

1) ***Build SDN network topology***: Use Mininet Python library to create simple network topology and configure IP and MAC addresses of Client, Server 1, and Server 2.

2) ***Development of SDN controller applications***:

Build a controller using the Ryu framework and manage traffic forwarding and flow table entries between network nodes to ensure inter-node communication and stable operation.

3) ***Connectivity test***:

Use ICMP ping to check the connectivity between Client, Server 1, and Server 2.

4) ***Create and load flow table entries***:

In the SDN switch, create and load the flow table entries. The controller builds the content of FlowMod through the `add_flow` method, and sets the idle timeout to 5 seconds.

5) ***Decision forwarding method and MAC learning***

Implement the MAC learning and port selection logic in the `packet_in_handler` event handling function

6) ***Traffic redirection and transparency effect***

Based on the source/destination IP addresses and TCP flags, determine whether a packet belongs to the configured `Client↔Server 1/Server 2` flow. If so, construct the corresponding match fields and use OpenFlow actions to rewrite the IP and MAC addresses, so that the connection is transparently redirected from Server 1 to Server 2.

7) ***Verify performance***

Using Wireshark, it was confirmed that it was indeed Server 2 that was responding. Meanwhile, from the Client's perspective, it was still communicating with Server 1.

D. Program Implementation

1) ***Forwarding***: The `RyuForward` application is implemented as a `RyuApp` and adheres to the OpenFlow 1.3 specification. Upon startup, it initializes the `mac_to_port` learning table for each switch and installs a low-priority table-miss entry. This enables unknown packets to be sent to the controller as `packet-in` events.

The primary forwarding logic is implemented in the `packet-in` processor, which parses the received Ethernet frames, disregards LLDP messages, and updates the `mac_to_port` table to learn the correspondence between MAC addresses and switch ports. Based on the learned state, the controller either selects a specific output port for unicast forwarding or resorts to flood forwarding.

For known destinations, it installs per-flow forwarding rules with an idle timeout of 5 seconds and forwards the current packet using `flow-mod` and `packet-out` messages. This allows subsequent packets of the same flow to be directly processed in the data plane without further controller intervention.

2) ***Redirection***: In the implementation of the redirection application, when the Client sends the TCP SYN packet to Server 1 for the first time, the switch triggers the `Packet_In` event to report to the controller. The controller then identifies in the `packet_in_handler` that this flow requires redirection. Subsequently, it installs a forward flow and a reverse flow: the forward flow matches packets from Client to Server 1, uses `OFPActionSetField` to rewrite the destination MAC/IP to that of Server 2, and outputs to the port where Server 2 is located; the reverse flow matches packets from Server 2 to Client, rewrites the source MAC/IP to that of Server 1, and outputs to the Client port. Both flows are configured with an `idle_timeout` of 5 seconds. For the first SYN packet, the controller simultaneously issues the flow table and constructs `OFPPacketOut`, rewriting the destination address according to the same rules and forwarding it to Server 2, thereby completing the connection establishment and enabling the Client side to transparently perceive the redirection process.

E. Difficulties and Solutions

1) ***Teamwork***:

When the team is dividing tasks, there may sometimes be a mismatch between the code version update and the report update, resulting in the report frequently containing a mixture of multiple code versions. Then the team adopted the real-time document strategy. By updating the code in real time in the shared document of the team, the members responsible for report writing could check at any time whether the code had been updated, ensuring a high degree of consistency between the report and the code.

2) ***Coding problem***:

At the beginning of writing the code, we encountered a series of issues such as the lack of separate handling for ARP, resulting in structural confusion; redundant MAC learning logic; incorrect setting of redirect condition judgment, etc. However, through multiple offline discussions among team members and consulting relevant materials, these difficulties were eventually overcame.

V. TEST AND RESULTS

This section presents the systematic testing methodology designed to fully validate the correctness, functionality, and per-

formance of the SDN-based traffic forwarding and transparent redirection system, strictly aligning with the task requirements specified in the project guidelines. All tests were conducted in a controlled Mininet emulation environment with a Ryu controller to ensure repeatability, accuracy, and compliance with the task specifications.

A. Test Environment and Setup

The testing was performed on a local development machine using Mininet to emulate the SDN network topology and Open vSwitch (OVS) as the data-plane switch. The controller logic was implemented using the Ryu framework with OpenFlow 1.3 support.

The topology consists of three hosts (one client, Server 1, Server 2) connected to a single SDN switch, with static IP/MAC configurations (Client: 10.0.1.5, Server 1: 10.0.1.2, Server 2: 10.0.1.3) as required. Before each test, existing Mininet instances were cleared (via `sudo mn -c`) to avoid port conflicts, ensuring a clean environment.

Fig. 4 verifies Task 1 (topology setup) via fixed IP assignments for all nodes. To further confirm Task 2 (basic connectivity), a full ping test was executed: as shown in Fig. 4, all nodes achieved zero packet loss, demonstrating that inter-node reachability is fully functional as required.

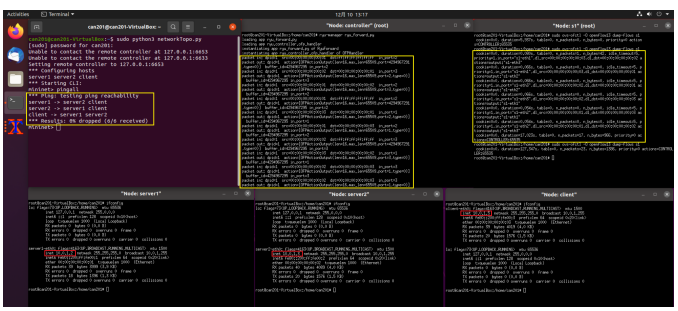

Fig. 4. Mininet topology with IP configurations and network ping test result

B. Testing Procedure

All experiments follow a fixed sequence:

1) Launch the topology via `networkTopo.py` (Task 1 implementation);

2) Start the Ryu controller application in either forward or redirect mode (corresponding to Task 4.1 and 5.1);

3) Run `server.py` on Server 1 and Server 2;

4) Execute `client.py` on the Client to initiate TCP connections, with a mandatory 5-second wait after ping tests (per Task 3) to ensure ICMP-induced flow entries are removed due to the 5-second idle timeout.

This strict order ensures that the observed behavior results solely from the controller logic, rather than unintended initialization effects.

C. Forwarding Functionality Test

To verify Task 4.1 (standard forwarding), the `ryu_forward.py` controller (implementing a learning-switch) was deployed. It dynamically installs flow entries based on observed traffic, as evidenced in the controller startup log (Fig. 5).

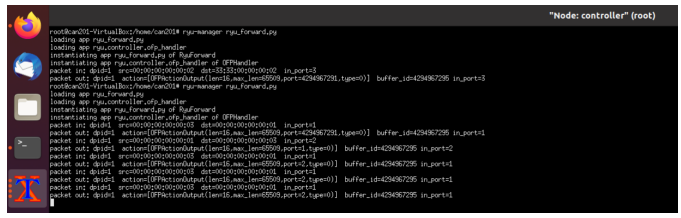

Fig. 5. Forwarding-mode controller startup

The client initiated a TCP connection to Server 1, with success criteria defined by Task 4.1: normal client-Server 1 communication and no traffic routed to Server 2. As shown in Fig. 6, the client and Server 1 exchanged application-layer messages seamlessly, while Server 2 remained inactive—confirming correct non-redirection behavior and full compliance with Task 4.1 requirements.

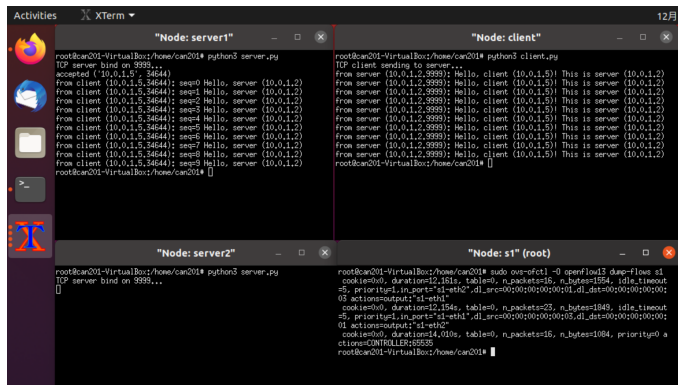

Fig. 6. Forwarding test result

D. Redirection Functionality Test

To evaluate Task 5.1 (transparent redirection), `ryu_redirect.py` controller was activated. It intercepts Client → Server1 TCP requests, rewrites packet headers, and redirects traffic to Server2 without the client's awareness—exactly as specified in the task. The controller log in Fig. 7 records the detection of the Client → Server1 SYN packet and the triggering of the redirection logic.

Validation was conducted via terminal outputs: the client reported connecting to Server 1, Server 2 processed the traffic, and Server 1 received no messages. As shown in Fig. 8, this result demonstrates the successful completion of Task 3 (traffic redirection) via transparent SDN control.

E. Latency Performance Analysis

To fulfill Task 4.2 and 5.2 (latency measurement), TCP three-way handshake latency was captured using Wireshark. Latency is defined as the time interval from the client's initial SYN packet to the final ACK packet (completing the three-way handshake), as specified in the tasks.

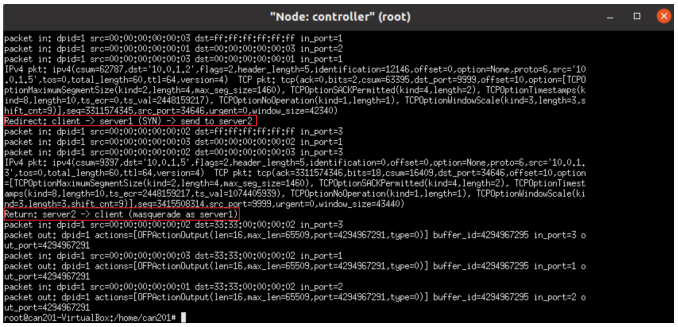

Fig. 7. Redirection-mode controller log

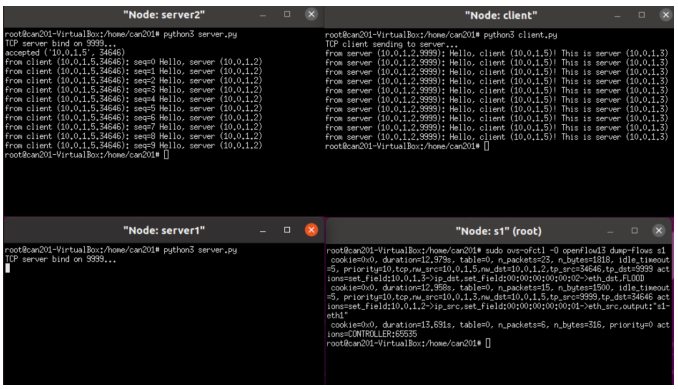

Fig. 8. Redirection test result

Figs. 9 and Figs. 10 display the Wireshark captures for forwarding and redirection modes, respectively. Fig. 11 (line chart) presents the mean latency over 10 trials for both scenarios: forwarding mode achieved an average latency of 0.0042 ms, while redirection mode yielded an average of 0.0077 ms. The latency difference is attributed to the additional packet header modification (IP/MAC rewriting) required for redirection, which introduces minimal processing overhead.

Fig. 11 (line chart) presents the mean latency over 10 trials for both forwarding and redirection. The forwarding mode (0.0042 ms) achieves a lower latency than the redirection mode (0.0077 ms). This difference is likely due to the additional processing required to modify the packet's destination address, which can introduce greater variability in forwarding time. Further automated experiments are necessary to validate this explanation and to minimize the influence of network fluctuations and other external factors.

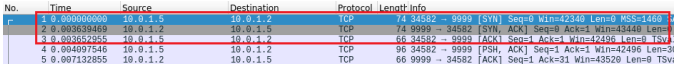

Fig. 9. Forwarding mode TCP handshake latency capture

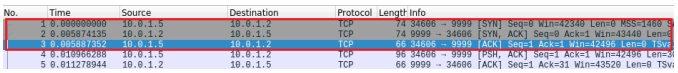

Fig. 10. Redirection mode TCP handshake latency capture

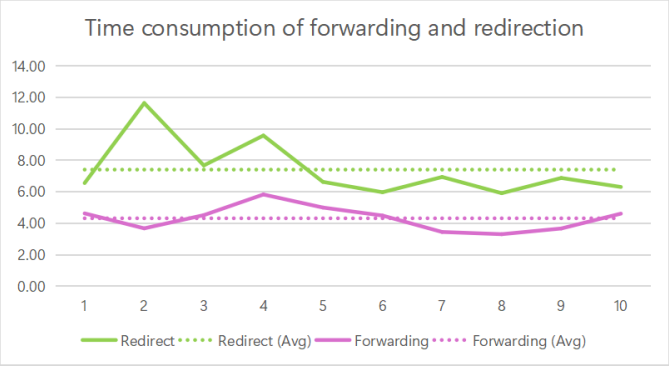

Fig. 11. Time Consumption of forwarding and redirection

F. Discussion

Experimental results validate the successful implementation of both standard forwarding and transparent redirection, fully meeting the project's core requirements. Functionality was rigorously verified through terminal outputs, controller logs, and host behavior—confirming that the designed controller logic produces consistent, repeatable data-plane behavior. The latency difference between forwarding (0.0042 ms) and redirection (0.0077 ms) is attributed to the additional packet header rewriting overhead in redirection, which aligns with expected performance tradeoffs for SDN-based traffic manipulation.

VI. Conclusion

Overall, this experiment successfully instantiated a software-defined networking (SDN) topology and demonstrated the feasibility of both traffic forwarding and traffic redirection. The empirical results indicate that, during connection establishment, redirection introduces higher latency relative to direct forwarding. These findings suggest that there is scope for further optimization and refinement. Future research should consider evaluating more complex and heterogeneous network topologies, as well as conducting repeated measurements under diverse and varying traffic load conditions, to obtain a more comprehensive and robust assessment of system performance.

Acknowledgment

Throughout the course of this project, all team members maintained active communication, participated in meaningful discussions, and fulfilled their individual responsibilities diligently. As a result, we conclude that every member contributed equally to the project.

References

[1] M. Alsaeedi, M. M. Mohamad, and A. A. Al-Roubaiey, "Toward Adaptive and Scalable OpenFlow-SDN Flow Control: A Survey," IEEE Access, vol. 7, pp. 107 346–107 379, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8784036/

[2] "nginx." [Online]. Available: https://nginx.org/en/

[3] H. Yang, H. Pan, and L. Ma, "A Review on Software Defined Content Delivery Network: A Novel Combination of CDN and SDN," IEEE Access, vol. 11, pp. 43 822–43 843, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10103706/